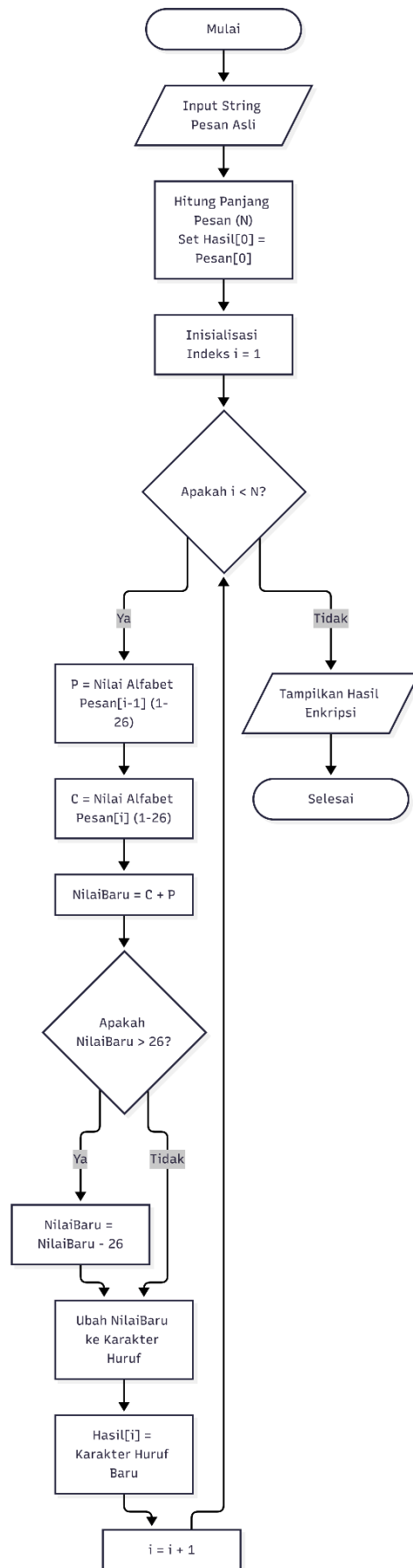




Penjelasan Logika Problem 2: Alien-In-The-Middle

Permasalahan *Alien-In-The-Middle* diselesaikan dengan menerapkan algoritma pergeseran karakter berantai (*sequential shift cipher*). Logika dasar dari sistem sandi ini bergantung pada posisi alfabetis dari setiap karakter dalam sebuah pesan teks. Pada tahap awal, pesan asli dibaca sebagai sekumpulan karakter, di mana karakter pertama tidak mengalami pergeseran sama sekali (nilai pergeserannya adalah 0). Mulai dari karakter kedua (indeks 1), setiap karakter akan digeser sejauh nilai alfabet dari karakter yang berada tepat di sebelahnya pada posisi sebelumnya ($i - 1$). Nilai alfabetis ini dipetakan secara konvensional, di mana karakter 'A' atau 'a' bernilai 1, 'B'/'b' bernilai 2, hingga 'Z'/'z' bernilai 26. Dengan memanfaatkan nilai karakter terdahulu sebagai kunci pergeseran dinamis, setiap huruf dalam string akan menghasilkan pergeseran unik yang saling bergantung satu sama lain.

Untuk menangani kasus di mana hasil penjumlahan pergeseran melebihi batas jumlah alfabet (lebih besar dari 26), algoritma menggunakan mekanisme perputaran (*wrapping*). Jika total penjumlahan nilai alfabet karakter saat ini dan nilai karakter sebelumnya menghasilkan angka lebih dari 26, maka nilai tersebut akan dikurangi 26 untuk mengembalikan urutan perhitungan ke awal alfabet ('A'). Setelah nilai numerik baru diperoleh, angka tersebut dikonversikan kembali menjadi bentuk karakter huruf yang bersesuaian. Proses iterasi pergeseran dan penyesuaian batas alfabet ini dilakukan secara berturut-turut untuk seluruh elemen string hingga karakter terakhir. Hasil akhir dari algoritma ini adalah sebuah string pesan terenkripsi baru yang aman dan tidak dapat dipahami secara langsung tanpa mengetahui pola pergeseran tersebut.

ALGORITMA TheLastAstronaut

KAMUS:

```
N, K, sisaAstronot, indeksAwal, targetIndeks,  
tereliminasi, i : INTEGER  
  
astronot : ARRAY [0..N-1] OF INTEGER
```

ALGORITMA:

```
INPUT N, K  
  
sisaAstronot <- N  
  
FOR i <- 0 TO N-1 DO  
    astronot[i] <- i + 1  
  
ENDFOR  
  
indeksAwal <- 0  
  
PRINT "Urutan Astronot yang Tereliminasi:"  
  
WHILE sisaAstronot > 1 DO  
    targetIndeks <- (indeksAwal + K - 1) MOD sisaAstronot  
    tereliminasi <- astronot[targetIndeks]  
    PRINT tereliminasi  
  
    IF tereliminasi MOD 2 == 0 THEN  
        K <- K + 2  
    ELSE  
        K <- K - 1  
    ENDIF  
  
    sisaAstronot <- sisaAstronot - 1  
    indeksAwal <- (indeksAwal + K) MOD sisaAstronot  
ENDWHILE
```

```
IF K < 2 THEN
```

```
    K <- 2
```

```
ENDIF
```

```
FOR i <- targetIndeks TO sisaAstronot - 2 DO
```

```
    astronot[i] <- astronot[i + 1]
```

```
ENDFOR
```

```
sisaAstronot <- sisaAstronot - 1
```

```
indeksAwal <- targetIndeks
```

```
ENDWHILE
```

```
PRINT "Astronot terakhir yang bertahan adalah nomor: ",  
astronot[0]
```

```
END ALGORITMA
```

Penjelasan Logika Problem 1: The Last Astronaut

Permasalahan The Last Astronaut diselesaikan menggunakan pendekatan simulasi eliminasi melingkar (circular elimination). Logika utama dari algoritma ini berfokus pada pelacakan posisi astronot yang aktif dalam lingkaran serta pembaruan variabel eliminasi K secara dinamis. Seluruh astronot diinisialisasi ke dalam sebuah array berukuran N dengan nomor identitas dari 1 hingga N . Untuk mensimulasikan bentuk lingkaran, algoritma menggunakan aritmetika modulo berdasarkan jumlah astronot yang masih tersisa (`sisaAstronot`). Posisi astronot yang akan dieliminasi pada setiap putaran ditentukan melalui rumus indeks $(\text{indeksAwal} + K - 1) \% \text{sisaAstronot}$. Pendekatan ini memungkinkan perhitungan melompati posisi secara berulang mengelilingi lingkaran tanpa perlu membuat struktur data linier yang kompleks secara fisik. Setelah astronot pada posisi target dieliminasi, elemen-elemen array setelahnya digeser ke kiri untuk menjaga kontinuitas urutan lingkaran.

Setelah seorang astronot tereliminasi, algoritma mengevaluasi sifat dari nomor identitas astronot tersebut untuk menentukan nilai K pada putaran berikutnya. Jika nomor astronot yang dieliminasi merupakan bilangan genap, maka nilai K ditambahkan sebanyak 2 ($K = K + 2$). Sebaliknya, jika nomor astronot tersebut merupakan bilangan ganjil, nilai K dikurangi 1 ($K = K - 1$). Algoritma juga dilengkapi dengan mekanisme validasi batas bawah, di mana jika hasil pengurangan menyebabkan nilai K kurang dari 2, maka nilai K secara otomatis dikembalikan menjadi 2. Posisi awal untuk perhitungan putaran berikutnya ditetapkan tepat pada indeks astronot yang baru saja tereliminasi. Proses perhitungan, pengeliminasian, dan penyesuaian nilai K ini diulang di dalam loop hingga hanya tersisa satu astronot. Hasil akhir yang dicetak oleh algoritma mencakup seluruh urutan astronot yang tereliminasi secara urut serta identitas astronot terakhir yang berhasil bertahan.